# Support Vector Machine for Image Classification using CIFAR-10 Dataset Neural Networks - Deep Learning Course

Computer Science Department,Aristotle University of Thessaloniki

## I. Introduction

This project addresses the foundational problem of image classification using one of the most widely adopted benchmark datasets, CIFAR-10. Consisting of 60,000 tiny 32x32 color images across 10 distinct object classes, CIFAR-10 serves as an ideal testbed for evaluating the performance of simple, yet conceptually powerful, classification algorithms and deep learning models.

In contrast to the previous project which focused on two prominent deep learning classifiers: the **Multilayer Perceptron (MLP)** and the **Convolutional Neural Network (CNN)**, where we highlighted their fundamental architectural differences and their respective strengths and weaknesses when applied to image data, this section demonstrates the implementation of a **Support Vector Machine** for the image classification of CIFAR-10.

### *Support Vector Machine (SVM)*

The Support Vector Machine is a supervised learning algorithm that classifies data by finding an optimal hyperplane which maximizes the margin between different classes. In the context of image classification, the SVM typically operates on extracted feature vectors rather than raw pixels, though it can use raw data:

- **Feature Vectors:** An SVM requires the input image to be represented as a vector. While it can accept flattened raw pixels (e.g., a 3072-dimensional vector for CIFAR-10), it is often more effective when combined with feature extraction techniques like HOG to reduce dimensionality and capture meaningful patterns before classification.
- **Margin Maximization:** The core principle is to find a decision boundary (hyperplane) that separates the classes with the widest possible gap (margin). It uses "support vectors"—the data points closest to the boundary—to define this separation.
- **Drawbacks for Images:** Standard linear SVMs struggle with the complex, non-linear patterns found in image data. To handle this, they require "kernel tricks" to map data into higher dimensions, which significantly increases computational complexity during training. Additionally, SVMs are inherently binary classifiers; extending them to multi-class problems (like the 10 classes of CIFAR-10) requires training multiple models (One-vs-Rest or One-vs-One).

By comparing the performance and efficiency of SVM with different kernels on image datasets, we highlight the trade-offs between defining explicit feature interactions (Linear, Polynomial) and leveraging infinite-dimensional feature spaces for maximum separability (RBF).

## II. The CIFAR-10 Dataset and Vector Preparation

The CIFAR-10 dataset is a widely adopted benchmark in computer vision, consisting of $60,000$ color images, each of size $32 \times 32$ pixels. These images are equally distributed across 10 distinct object classes (e.g., airplane, automobile, bird, cat, dog), with $6,000$ images per class. The dataset is conventionally split into two subsets:
Training Set: $50,000$ images ($N_{\text{train}}$) used to learn the structure of the data.
Test Set: $10,000$ images ($N_{\text{test}}$) used for final evaluation.

A key challenge of CIFAR-10 is the low resolution ($32 \times 32$), which forces NNs to rely on fine details, and the high intra-class variance (e.g., different angles and lighting for the same object), making generalization difficult for shallow models. Vector Preparation for Multi-Layer Perceptron and Convolutional Neural Network:

**Flattening**: It is necessary for the Support Vector Machine. Each $32 \times 32$ color image has three color channels (Red, Green, Blue). To convert the image into a single feature vector $\mathbf{x}$, the dimensions are multiplied:

$$D = 32 \times 32 \times 3 = 3072 \text{ features}$$

Every image is thus represented by a 3072-dimensional vector. This process is necessary because an SVM's input layer expects a single vector for each sample.

**Normalization:** The raw pixel intensity values range from 0 to 255. To prevent features with inherently larger magnitude (such as those in the blue channel, if not properly scaled) from producing mathematically massive distances and thus "break" the formulas used to calculate similarity, this preprocessing step is required for the SVM, as it is distance-based algorithm. For example, pixel values range from 0 to 255 and a difference in brightness between two images results in a mathematically massive distance (e.g., $200^2$), even if the images are visually similar. The large pixel values lead to larger gradients. The Sequential Minimal Optimization algorithm used in SVM libraries we utilize, fails to converge without normalization. With unnormalized data, the steps SMO takes become tiny and ineffective. It has to zig-zag millions of times across

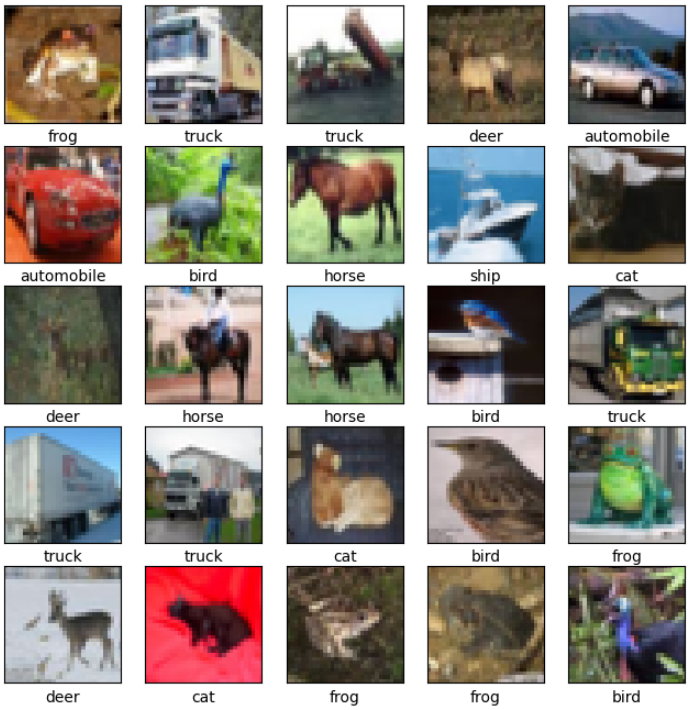



Fig. 1. Samples from every class of CIFAR-10.

the "canyon" floor to reach the solution that it could have reached in just a few steps on a "bowl-shaped" surface. Thus, normalization is a critical step as it helps accelerate convergence during training. This is achieved by converting the data type to float and dividing all pixel values by $255.0$, resulting in feature vectors where all components lie in the normalized range $[0.0, 1.0]$. This step is crucial for ensuring stable and efficient training of the Support Vector Machine.

### *Training Algorithm for Support Vector Machine*

The training of a Support Vector Machine is formulated as a convex Quadratic Programming (QP) problem, distinct from the iterative error minimization used in neural networks. In its primal form, the objective is to minimize the squared norm of the weight vector, represented as:
$\min_{w,b} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i$, subject to the linear constraints $y_i(w \cdot x_i + b) \geq 1 - \xi_i$.
This equation seeks the "flattest" decision boundary (maximizing the margin) while penalizing misclassifications ($\xi_i$) based on the regularization parameter $C$. However, to enable the use of non-linear kernels, standard libraries solve the Dual QP problem instead. This transforms the minimization task into maximizing the Lagrangian objective:

$$\text{maximize } L(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

subject to the constraints $0 \leq \alpha_i \leq C$ and $\sum \alpha_i y_i = 0$.
This dual formulation isolates the data representation into the kernel function $K(x_i, x_j)$, allowing the algorithm to operate in high-dimensional feature spaces without explicit computation. The critical drawback of this QP approach is that the objective function involves a dense matrix of pairwise interactions between all training samples ($n \times n$), leading to a computational complexity between $O(n^2)$ and $O(n^3)$. This scaling issue necessitates the use of efficient decomposition algorithms, such as Sequential Minimal Optimization (SMO), which breaks the massive QP problem into analytically solvable sub-problems of just two variables at a time. This happens by employing decomposition, so instead of optimizing the entire vector of Lagrange multipliers ($\alpha$) simultaneously, which scales poorly with the large CIFAR-10 dataset, SMO breaks the problem down into the smallest possible sub-problems consisting of just two variables. This pairwise approach is strictly necessary due to the linear equality constraint $\sum y_i \alpha_i = 0$; modifying a single multiplier would violate this balance, so a second multiplier must always be adjusted in tandem to compensate. In each iteration, the algorithm uses heuristics to select two multipliers that violate the Karush-Kuhn-Tucker (KKT) conditions, fixes all other variables, and then solves for the optimal values of the chosen pair analytically. Because this reduced problem is simply a quadratic function restricted to a line segment (to satisfy $0 \leq \alpha_i \leq C$), the update can be computed instantly using basic algebra without matrix inversion. This process repeats until the model converges, allowing SVMs to be trained on large datasets with linear memory complexity.

### ***One-vs-Rest Classification Approach***

As we know, SVM models were originally designed to solve binary classification problems, they require modification when extended to multiclass problems like CIFAR-10. The two primary approaches discussed are **One-vs-One** and **One-vs-Rest**.In the One-vs-One approach, a separate SVM model is trained for every possible pair of categories. This means that for a problem with $k$ classes, $\frac{k(k-1)}{2}$ individual models are created. During the prediction phase, each model "votes" for the category it determines the sample belongs to, and the final classification is decided by which category receives the most votes.Conversely, the One-vs-Rest approach involves training one separate SVM model for each category to distinguish it from all other remaining categories. Consequently, for a problem with $k$ categories, only $k$ models are created. During prediction, the model that yields the highest "confidence" (or the highest probability) determines which category is assigned to the sample.For a specific problem with 10 categories, such as CIFAR-10, the One-vs-One approach would require training 45 separate models, whereas the One-vs-Rest approach requires only 10 models. Due to the need for computational efficiency and resource conservation, the One-vs-Rest approach was selected.

## III. Support Vector Machine with Linear Kernel

The linear kernel represents the most fundamental approach to Support Vector Classification, defined mathematically as the simple dot product:
$K(x_i, x_j) = x_i^\top x_j$.
It operates on the assumption that the classes are linearly separable—meaning a single, flat hyperplane can cleanly divide the data. While this computational simplicity makes it incredibly fast and less prone to overfitting in scenarios where the number of features dwarfs the number of samples , it lacks the flexibility to model the intricate, curved decision boundaries often required for complex pattern recognition.When applied to the CIFAR-10 dataset, the limitations of this linearity become starkly evident. Since the model treats the flattened 3,072-pixel vector as a coordinate in space, a linear SVM essentially performs "template matching." It learns a single, average prototype for each class—for instance, the "car" weight vector might just look like a blurry red blob because many cars in the training set happen to be red and centered. Consequently, the model struggles to generalize; a green car in the corner of an image would likely be misclassified because it does not align geometrically with the rigid, average template the model has learned. This inability to capture non-linear relationships or translation invariance results in significantly lower accuracy compared to models that can learn hierarchical features, such as CNNs.

**Model Architecture:**

- **Preprocessing**: The first step applies Z-score normalization, mathematically defined as $1z = \frac{x-\mu}{\sigma}$.2 The pipeline first applies standard scaling to normalize pixel values to a mean of 0 and variance of 1. In the context of raw images (where pixels range from 0 to 255), this transformation is critical to prevent feature dominance and improve the optimization procedure as we discussed earlier. Thus the SVM treats all features equally and converges quickly.
- **Dimensionality Reduction**: The second step addresses the **Curse of Dimensionality**. A raw CIFAR-10 image has 3,072 dimensions (features). In such high-dimensional spaces, data points become incredibly sparse, making it statistically difficult to find a robust separating hyperplane without overfitting. The parameter n_components=0.95 instructs the algorithm to calculate the eigenvalues of the covariance matrix of the features and keep only the top $k$ eigenvectors, which are the principal components required to preserve 95% of the original information (variance). The algorithm discards noise and redundant pixels, keeping only the principal features that account for 95% of the image variance. This shrinks the input vector size and speeds up training. For CIFAR-10 this reduces the feature count from 3,072 to fewer than 200 components.
- **Pipeline**: The use of the Pipeline object is a practice for preventing data leakage. If one were to apply PCA or Scaling to the entire dataset before splitting it into training and testing sets, the model would "peek" at the test data's distribution (its mean and variance). By wrapping these steps, scikit-learn ensures that the scaler and PCA matrix are fitted only on x_train, and those exact parameters are then applied blindly to transform x_test.
- **Primal Optimization**: This is the most critical performance parameter dual=False in this specific pipeline. It tells scikit-learn to solve the Primal optimization problem directly rather than the Dual problem.t with the highest probability is the network's prediction. It matters here, because the number of training samples $N = 50,000$ is much higher than the final number of features $D \approx 200$ arised from the PCA. Solving the Dual problem (which scales with $N$) would be computationally wasteful. Solving the Primal problem (which scales with $D$) is drastically faster and uses less memory.
- **Regularization Parameter**: The parameter $C$ controls the penalty for misclassification. It determines the trade-off between finding a wider margin which is our optimization problem and correctly classifying every single training point. The low value of C=2 points to a **Soft Margin**, meaning allows misclassifications in exchange for a wider, simpler decision boundary, in order to reduce overfitting.

**Training Analysis and Evaluation:**
The model's performance was rigorously evaluated using Squared Hinge Loss, mathematically defined as $L(y, f(x)) = \max(0, 1 - y \cdot f(x))^2$. The second term measures how "safe" the prediction is, where f is the models' prediction score. If it's positive, the point is either inside the margin or on the wrong side. This function squares the error term, which serves a specific computational purpose: it transforms the loss landscape into a smooth, differentiable surface by removing the sharp "kink" found in the standard Hinge Loss at the margin. The practical consequence of this choice is that the model penalizes outliers—points significantly on the wrong side of the margin—quadratically rather than linearly, making this classifier more sensitive to noisy data points.

The accuracy of roughly **40%** for a Linear SVM on the CIFAR-10 dataset highlights the fundamental mismatch between the model's simplicity and the data's complexity. First, while it is better than the random baseline loss for a 10-class problem (which is $\approx 10\%$), it remains far from an acceptable level for modern image classification, indicating the model is only marginally learning the task. CIFAR-10 images are 3,072-dimensional vectors (flattened $32 \times 32 \times 3$ pixel grids) where classes like "cat" or "airplane" are defined by complex, non-linear manifolds—meaning their patterns curve and twist in high-dimensional space rather than sitting on opposite sides of a flat sheet. A Linear SVM, by definition, attempts to slice this space with a single, flat hyperplane (a straight line in 2D). It effectively relies on "template matching," learning a blurry average representation for each class (e.g., a "frog" template might just be a green blob in the center). This approach fails to capture invariances; if an object is shifted, rotated, or has a different background color than the average, the linear dot product score drops drastically. Furthermore, raw pixel data is highly sensitive to lighting and texture variations that a linear boundary cannot disentangle. While 40% is significantly better

than random guessing, it plateaued there because the model simply lacks the mathematical capacity for non-linearity to distinguish between classes that overlap in the linear pixel space.

In a Soft-Margin SVM, any point that is on the margin or violating the margin is a support vector. Mathematically, these are points where the decision function output is $\leq 1$. In order to evaluate our model's performance even more, we calculated the total number of support vector machines needed to define the margin. With utility of **One-vs-Rest** approach, the model trained 10 binary classifiers (a specific class vs anything else) we resulted with a total of 34.878 support vectors per classifier. This means that for the any classifier, roughly 35,000 out of 50,000 images (70%) are considered support vectors. Because 70% is needed to define the decision boundary our low accuracy ( 40%) is confirmed. The model effectively thinks nearly every image is a confusing "edge case" because it is trying to cut a curved manifold with a "straight knife".

**Confusion Matrix Analysis:**
The confusion matrix provides a granular diagnosis of the 40% accuracy score, visually confirming the limitations of the Linear SVM on CIFAR-10. The diagonal values (representing correct classifications) are highest for vehicles (Classes 0, 1, 8, 9: Airplane, Auto, Ship, Truck), with scores like 575 for Class 6 and 568 for Class 8. In contrast, the model struggles significantly with animals (Classes 2, 3, 4, 5: Bird, Cat, Deer, Dog), where the diagonal counts are much lower (e.g., only 194 correct for Bird and 195 for Cat).

**The Color Trap**:
The matrix exposes that the linear model is likely classifying based on background color rather than shape. It proves that the Linear SVM is essentially acting as a "background color detector" performing decently on distinct environments (roads, sea, sky) but failing completely when objects share similar colors or organic shapes.
Planes vs. Ships (0 vs. 8): There is massive confusion between Class 0 and Class 8 (e.g., 207 Airplanes were misclassified as Ships [Row 0, Col 8]). This occurs because both classes typically feature large amounts of blue (sky vs. ocean), and the linear kernel cannot distinguish the texture of water from air.
Birds vs. Frogs (2 vs. 6): A striking error appears in Row 2, where 188 Birds were misclassified as Frogs (Class 6)—nearly as many as were correctly identified (194). This suggests the model is conflating the green of trees (birds) with the green of frogs.
Cats vs. Dogs (3 vs. 5): The model heavily confuses these four-legged animals (e.g., 184 Cats misclassified as Dogs), likely because their color palettes and textures overlap significantly in a linear pixel space.

```
Training SVM with Linear Kernel
Elapsed time 176.5 seconds
Final Accuracy:  0.4065
```

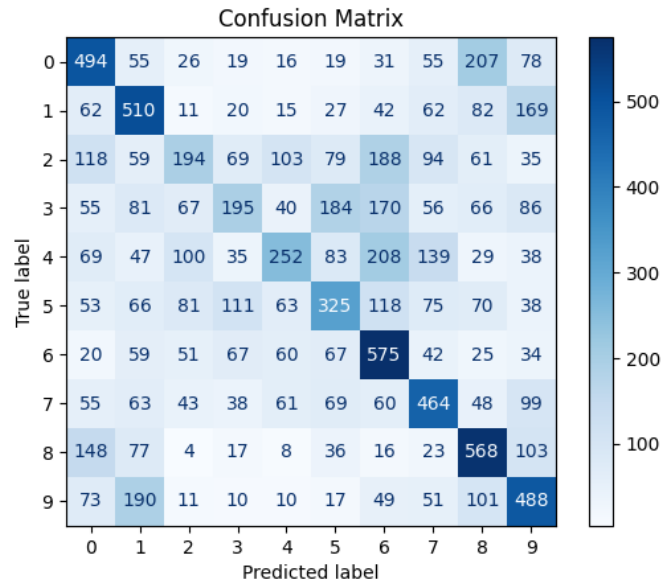



Fig. 2. Confusion Matrix for Linear Kernel.

## IV. Support Vector Machine with Polynomial Kernel

The Polynomial Kernel represents a step up in complexity from the linear approach, allowing the SVM to model curved decision boundaries by explicitly calculating interactions between input features. Mathematically defined as $K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$, this kernel projects the data into a higher-dimensional space where the "coordinates" are combinations of the original pixel values. The critical parameter is the degree ($d$). In the context of CIFAR-10, a degree of $d = 2$ tells the model not just to look at "Pixel A" and "Pixel B" separately, but to consider their product ($PixelA \times PixelB$). This allows the model to learn simple non-linear relationships, such as "if Pixel A is dark AND Pixel B is light, this is an edge". As this parameter scales, the risk of overfitting is increased. The Polynomial Kernel can bend and twist the decision boundary to fit the "clouds" of data classes more tightly. This typically results in higher accuracy than the 40% baseline of the linear model, as it can distinguish objects that are not linearly separable.

### *Model Architecture*

- **Preprocessing**: The first step applies Z-score normalization, mathematically defined as $z = \frac{x-\mu}{\sigma}$. The pipeline first applies standard scaling to normalize pixel values to a mean of 0 and variance of 1. In the context of raw images (where pixels range from 0 to 255), this transformation is critical to prevent feature dominance

and improve the optimization procedure as we discussed earlier. Thus the SVM treats all features equally and converges quickly.

- **Dimensionality Reduction**: The second step addresses the **Curse of Dimensionality**. A raw CIFAR-10 image has 3,072 dimensions (features). In such high-dimensional spaces, data points become incredibly sparse, making it statistically difficult to find a robust separating hyperplane without overfitting. The parameter n_components=0.90 instructs the algorithm to calculate the eigenvalues of the covariance matrix of the features and keep only the top $k$ eigenvectors, which are the principal components required to preserve 90% of the original information (variance). The algorithm discards noise and redundant pixels, keeping only the principal features that account for 90% of the image variance. This shrinks the input vector size and speeds up training. For CIFAR-10 this reduces the feature count from 3,072 to fewer than 200 components.
- **Pipeline**: The use of the Pipeline object is a practice for preventing data leakage. If one were to apply PCA or Scaling to the entire dataset before splitting it into training and testing sets, the model would "peek" at the test data's distribution (its mean and variance). By wrapping these steps, scikit-learn ensures that the scaler and PCA matrix are fitted only on x_train, and those exact parameters are then applied blindly to transform x_test.
- **Regularization Parameter**: The parameter $C$ controls the penalty for misclassification. It determines the trade-off between finding a wider margin which is our optimization problem and correctly classifying every single training point. The high value of C=10 points to a **Hard Margin**, meaning the model is strict. It places a huge penalty on any point being on the wrong side. It will twist and turn the decision boundary frantically to ensure every single training point is classified correctly. Even though this selection can be prone to overfitiing, due to higher PCA , is considered a safe choice.
- **Polynomial Degree**: When setting the polynomials' degree d=2, we are telling the SVM to look at pairs of features, not individually. By checking these pairwise interactions, the SVM tries to match global shapes or templates rather than just average colors.
- **Inhomogeneous Kernel**: Setting coef0=1 is a crucial design choice that changes polynomial kernel from "homogeneous" to "inhomogeneous," and boosts the model's flexibility. Mathematically, it adds a bias term to the kernel equation $(\gamma\langle x, y\rangle + r)^d$, which forces the expansion to include not just the complex quadratic interactions (like pairs of pixels) but also the original lower-order linear features the raw pixel values themselves. Without this parameter the SVM would ignore the simple linear relationships, such as whether a specific pixel is bright or dark and only focus on complex curvature. By setting it to 1, we ensure the classifier can draw simple linear boundaries where appropriate while still utilizing the quadratic terms to capture more complex shapes.

**Training Analysis and Evaluation**

The jump to 54.20% accuracy with the Polynomial Kernel ($d = 2$) marks a decisive shift from simple template matching to capturing structural complexity. Unlike the linear model's rigid reliance on a flat hyperplane, the quadratic kernel implicitly projects the input data into a vastly larger feature space of pixel interactions, allowing the SVM to construct curved decision boundaries—specifically, quadric hypersurfaces. This enables the classifier to learn not just that "frogs are green" (a linear trait), but that specific pairs of pixel intensities and texture gradients define a frog, effectively modeling correlations between features that a linear model ignores. The crucial addition of coef0=1 ensures the model retains the original linear signal while layering on this curvature, offering a best-of-both-worlds approach. However, the plateau at roughly 54% reveals the inherent ceiling of shallow learning on raw pixels: despite the added non-linearity, the model remains brittle to spatial variations.

For this **Hard Margin** problem where we set the penalty C=10, the overfitting is expected. Due to this, our classifier resulted in total 40.050 support vectors to shape the margin. This high number of support vectors that lie inside the margin or on the wrong side of the hyperplane indicate that the model is defining its boundary not by a few informative edge cases, but by the dense, noisy interior of the overlapping class distributions. The Linear Kernel gave 40% of accuracy, but the polynomial kernel with extra 2,000 support vectors resulted up to 14% higher accuracy, which is an acceptable trade-off.

**Confusion Matrix Analysis**

The confusion matrix reveals that while our model has grasped broad semantic categories—effectively separating "Machines" from "Animals"—it struggles significantly with fine-grained distinctions where feature overlap is high. The strong diagonal values for classes like Ships and Automobiles indicate that the SVM can easily latch onto distinct geometric shapes and consistent backgrounds, but the specific off-diagonal errors expose the limitations of raw pixel learning. For instance, the high confusion between "Trucks" and "Automobiles" (175 errors) suggests the model cannot distinguish between their shared boxy, metallic structures, while the heavy mix-up between "Cats" and "Dogs" (over 160 errors each way) reflects an inability to resolve subtle differences in fur texture and pose. Additionally, the frequent confusion between Airplanes and Ships (over 100 errors) points to a reliance on background color, where the model likely conflates the blue pixels of the sky with those of the ocean. Ultimately, the classifier has learned the general "templates" of these objects but lacks the translation invariance and local feature detection required to disentangle classes that look visually similar in the PCA-reduced space.

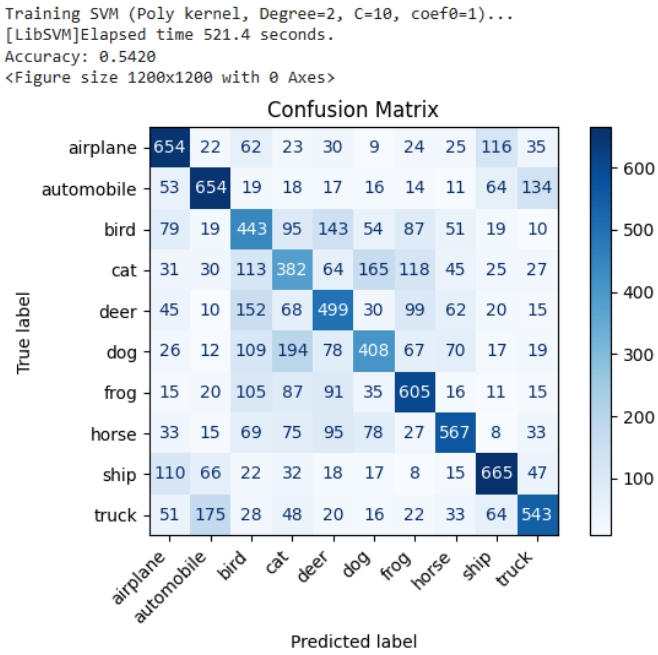



Fig. 3. Confusion Matrix for Polynomial Kernel.

## V. Support Vector Machine with RBF Kernel

The Radial Basis Function (RBF) kernel represents a distinct paradigm shift in Support Vector Classification, defined mathematically as a function of Euclidean distance:

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

Unlike the linear kernel's assumption of flat separability, the RBF kernel operates on the principle of similarity, implicitly mapping the input data into an infinite-dimensional feature space where complex, non-linear boundaries become linear. While this theoretical capacity allows it to model virtually any class distribution given enough data, its application to the CIFAR-10 dataset exposes the treacherous nature of high-dimensional pixel spaces. Since the kernel relies entirely on the Euclidean distance ($||x_i - x_j||$) between images, an RBF SVM essentially functions as a sophisticated "soft nearest-neighbor" classifier. Instead of learning a single, average prototype for a "car" it memorizes local clusters of training examples, classifying a new image based on its pixel-wise proximity to these stored instances. However, this reliance on raw pixel alignment is its undoing; the Euclidean distance between two images of the same dog can be massive if one is simply shifted by a few pixels or slightly darker, while the distance between a dark dog and a dark truck might be deceptively small. Consequently, the model suffers from the **curse of dimensionality** where the notion of distance loses statistical meaning in 3,072-dimensional space, failing to capture the structural or semantic invariances.

### *Model Architecture*

- **Preprocessing**: The first step applies Z-score normalization, mathematically defined as $z = \frac{x-\mu}{\sigma}$. The pipeline utilizes StandardScaler to normalize all feature values to a mean of 0 and a variance of 1. Because SVMs rely on calculating the Euclidean distance between data points to define the margin, features with large numeric ranges would otherwise disproportionately influence the model. This transformation ensures the SVM treats all features equally and significantly improves the convergence speed of the optimization algorithm.
- **Dimensionality Reduction**: The second step addresses the Curse of Dimensionality and computational efficiency. The parameter n_components=0.95 inside the PCA instructs the algorithm to compute the eigenvalues of the covariance matrix and retain only the minimum number of principal components required to preserve 95% of the original data's variance. By discarding the dimensions responsible for the lowest 5% of variance which is noise, this step reduces the feature vector size and computational volume.
- **Regularization Parameter**: The value of C=10 is chosen to be relatively high in order to instruct the model to penalize misclassified training points strictly. It prioritizes a **Hard Margin** that fits the training data closely, accepting a higher risk of overfitting in exchange for lower bias, compared to a lower $C$ which would allow more errors for a wider, smoother decision boundary.
- **Kernel Coefficient**: This parameter gamma='scale' controls the "reach" of a single training example. A "scaled" gamma is calculated as $1/(n_{features} \cdot X.var())$. This allows the model to automatically adjust the curvature of the decision boundary based on the feature variance, ensuring the bell-shape of the RBF is neither too narrow (overfitting) nor too wide (underfitting).
- **One-vs-Rest**: The decision_function_shape='ovr' the One-vs-Rest strategy. Since SVMs are inherently binary classifiers, this parameter enables multi-class classification by training one classifier per class (fitting class $A$ against all other classes combined).

### *Training Analysis and Evaluation*

The progression to 57.13% accuracy with the RBF kernel represents the upper bound of performance for this non-convolutional architecture, offering a modest but meaningful improvement over the Polynomial kernel. While the Polynomial kernel ($d = 2$) captured global structural interactions, the RBF kernel operates on local proximity, implicitly projecting the data into an infinite-dimensional space. By defining similarity through the exponential decay of Euclidean distance ($\exp(-\gamma ||x - x'||^2)$), the RBF kernel shifts the learning paradigm from global algebraic surfaces to local topological mapping. This capacity to model local distributions explains the performance increment, as it better accommodates the non-linear intra-class variance of the CIFAR-10 dataset. However, the performance plateau reaffirms that without the hierarchical feature extraction and translation invariance of Convolutional Neural Networks (CNNs), the model remains sensitive to the "curse of dimensionality" and pixel-level noise.

### *Confusion Matrix Analysis*

The localized learning mentioned above is clearly reflected in the Confusion Matrix, which shows the model has learned to separate broad semantic super-categories. There is a distinct ”block” of confusion among vehicles (Automobile/Truck) and a separate block among animals (Bird/Cat/Dog/Deer), with very little crossover between the two groups. For example, the model rarely confuses a ’Frog’ with a ’Truck’ (only 11 errors), proving it has mastered distinguishing low-level feature variances like metallic/geometric versus organic/textured. However, the specific ”soft” overlaps reveal the limitations of relying on raw pixel data (even with PCA). The massive confusion between Cats and Dogs (where 166 cats were misclassified as dogs) suggests the RBF kernel struggles to disentangle classes that share similar texture gradients and color histograms. Without the translation invariance of a CNN, a dog facing left looks statistically distinct from a dog facing right, confusing the distance-based RBF metric.

```
Training SVM with RBF kernel (C=10, gamma=scale)...
[LibSVM]Elapsed time: 595.6 seconds.
Evaluating model...
Accuracy: 0.5713
<Figure size 1200x1200 with 0 Axes>
```

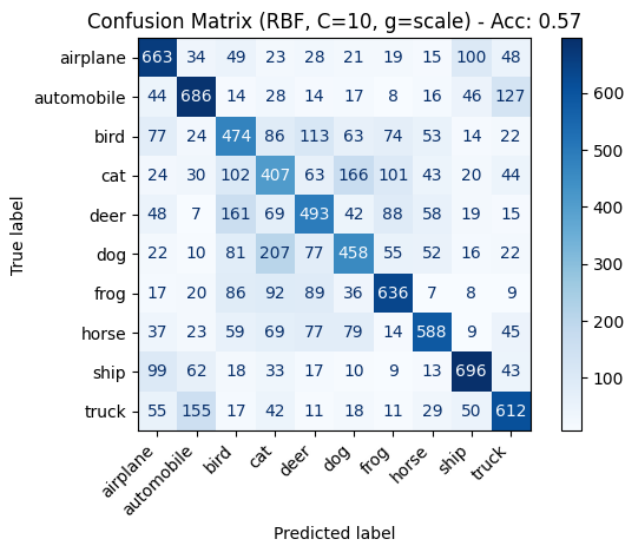



Fig. 4. Confusion Matrix for RBF Kernel.

## VI. Feature Engineering with Histogram of Oriented Gradients

The transition to HOG represents a fundamental shift in how the machine ”perceives” the image: moving from photometric perception such as brightness of a pixel to geometric perception that refers to shape construction.

The HOG algorithm operates through a four-stage signal processing pipeline designed to distill raw noise into structural signal. For every pixel, the algorithm calculates the gradient vector $(magnitude, direction)$ by comparing it to its neighbors. Mathematically, if pixel $(x, y)$ has intensity $I(x, y)$, the gradient in $x$ is $G_x = I(x+1, y) - I(x-1, y)$. This eliminates constant colors. A flat white wall has a gradient of 0. A sharp edge has a high magnitude. The model no longer sees colors, such as white but it sees shapes like a vertical line instead. Then the image is divided into small grid cells (e.g., $8 \times 8$ pixels). Instead of storing 64 individual gradient vectors, HOG aggregates them into a histogram of 9 orientations (e.g., 0°, 20°, 40°...). This grants local translational invariance. If a car’s wheel shifts by 2 pixels, the raw pixel values change completely, but the ”vertical edge” vote still falls into the same $8 \times 8$ cell. The histogram remains stable even if the object shifts slightly. Eventually, cells are grouped into larger ”blocks” (e.g., $2 \times 2$ cells), and the vector is normalized-divided by its magnitude.

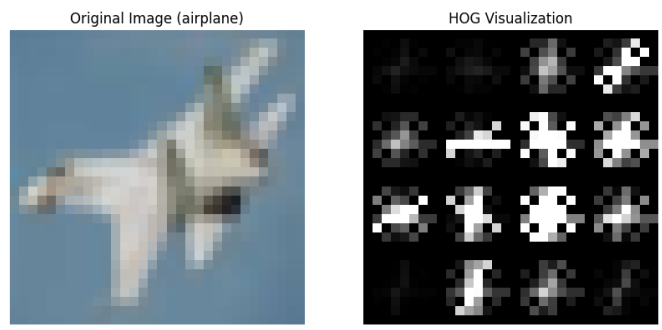



Fig. 5. Caption

### *Justification*

SVMs (especially with RBF kernels) rely on Euclidean distance to measure similarity. In raw pixel space, the Euclidean distance is dominated by background color and lighting. In HOG space, the distance represents structural deviation. A ”Truck” is structurally distant from a ”Frog” because their edge histograms occupy orthogonal regions of the vector space, making the separation margin $(2/||w||)$ much easier for the SVM to maximize. Although HOG is high-dimensional (often thousands of features), it is dense with information. Raw pixels are high-dimensional but ”sparse” in information (mostly noise and background). HOG compresses the image into a manifold where the axes represent meaningful concepts (vertical edges, horizontal curves) rather than arbitrary sensors (pixel locations).

### *Training Analysis and Evaluation*

In this final SVM model, the feature engineering proved to be ideal. The significant leap to 64% accuracy marks the decisive advantage of structural feature engineering over raw pixel learning. While the previous RBF model operating on raw pixels plateaued at 57%, the introduction of Histogram of Oriented Gradients (HOG) has successfully pushed the decision boundary beyond the shallow learning ceiling. This 7% improvement confirms that the model is no longer struggling to find correlations in noisy pixel intensities but is instead classifying based on robust geometric silhouettes.The elevation of global accuracy to 64.00% validates the hypothesis that structural feature engineering offers a superior signal-to-noise ratio compared to raw pixel learning.

```
Training SVM with RBF kernel, C=10, gamma=scale)...
Evaluating model...
Accuracy: 0.64
Global Accuracy: 0.64
```

The per-class accuracy reveals a stark dichotomy in how the model perceives the world, splitting strictly between rigid mechanical objects and deformable organic subjects: The model achieves its highest performance on vehicles: Automobile 77.0%, Truck 73.9%, and Airplane 73.7%. These classes are defined by standardized, rigid geometries (straight lines, wheels, wings) that produce consistent, high-magnitude gradient histograms. HOG excels here because a car's edge distribution remains statistically stable regardless of its color. Conversely, the model struggles significantly with animals, particularly Cats 44.6% and Dogs 51.6%. Unlike cars, these biological entities are "deformable"—a cat can curl into a ball, stretch, or sit, drastically altering its edge histogram. Furthermore, distinguishing a cat from a dog often relies on texture rather than distinct shape outlines. Since HOG discards texture to focus on edges, the model lacks the information density required to separate these classes.

```
--- Per-Class Accuracy ---
airplane    : 73.70%
automobile  : 77.00%
bird        : 50.80%
cat         : 44.60%
deer        : 57.20%
dog         : 51.60%
frog        : 70.70%
horse       : 65.60%
ship        : 70.60%
truck       : 73.90%
```

Fig. 6. Per Class Accuracy

The RBF kernel is now operating on a feature space defined by edge orientations rather than luminosity. This effectively grants the SVM a degree of photometric invariance; it no longer cares if a car is red or blue, only that it has the rectangular gradient structure of a chassis. This is a critical step forward, as the kernel can now construct decision "bubbles" around shape archetypes (e.g., the specific gradient distribution of a "Plane" vs. a "Bird") without being distracted by background color noise.

### *Confusion Matrix Analysis*

The confusion matrix reveals a distinct bifurcation in model performance governed by morphological stability, where rigid, mechanical classes like Automobiles 77.0% and Trucks 73.9% significantly outperform deformable biological classes. This divergence aligns with the HOG descriptor's reliance on fixed geometric edges; vehicles possess standardized, rectilinear silhouettes that map to distinct regions in the feature space, whereas animals like Cats 44.6% and Dogs 51.6% exhibit high intra-class variance due to flexible poses and a reliance on texture rather than outline. Consequently, the matrix displays a dense block of "semantic confusion" between these two species, with 162 cats misclassified as dogs and 165 dogs as cats, confirming that their gradient histograms are nearly identical without color or textural context to separate them. While Frogs 70.7% defy this trend due to their compact, blob-like consistency which minimizes gradient variation, the remaining errors—such as the 81 Trucks misclassified as Automobiles—highlight the semantic gap where the model effectively captures the general "vehicle" shape but lacks the granular feature resolution to distinguish subclass variations (like a sedan vs. a pickup) that share similar gradients.

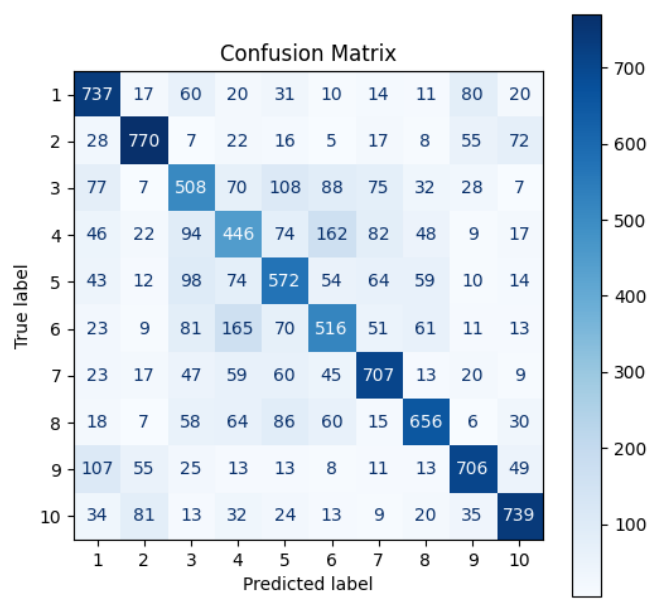
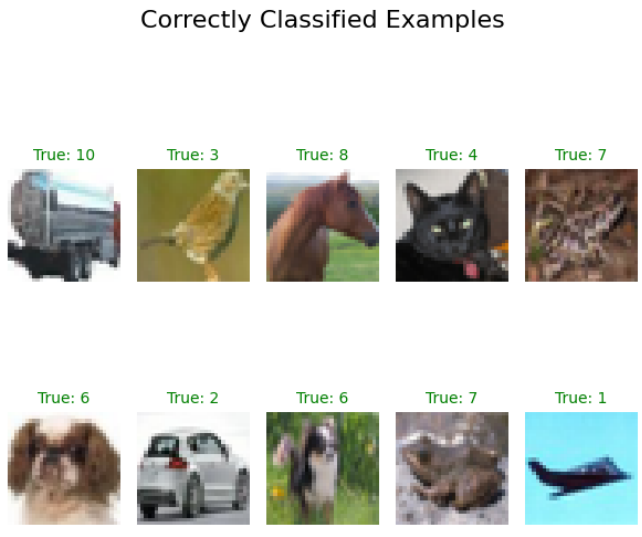
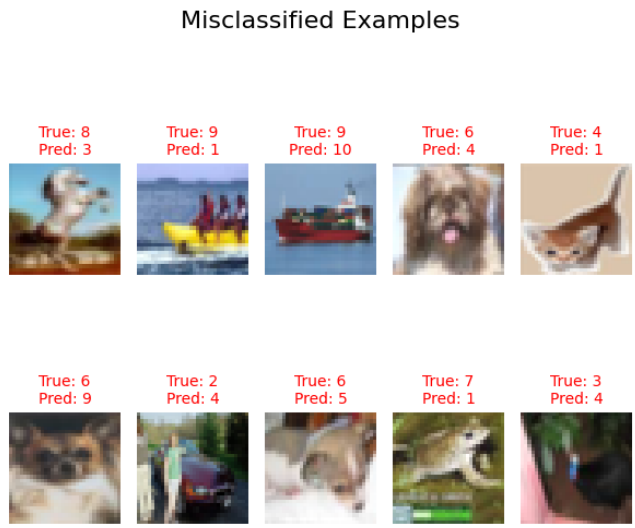



Fig. 7. Confusion Matrix for RBF Kernel with HOG




## VII. Feature extraction with Convolutional Neural Network

In order to enhance our model's performance we are combining a Convolutional Neural Network with a Support Vector Machine using a Radial Basis Function (RBF) kernel as a hybrid approach. This leverages the CNN's superior ability to extract spatial features and the SVM's ability to find an optimal separating hyperplane in a high-dimensional space.

### *Feature Extraction*

- **Data Augmentation:** The model utilizes aggressive data augmentation, implemented via the ImageDataGenerator, to enhance the robustness and generalization capabilities of the network, mitigating the risk of overfitting on the training set. The defined transformations include random rotation up to $25°$, horizontal and vertical shifts up to $60\%$ of the image dimensions, a $60\%$ shear intensity, and up to $60\%$ zoom. Crucially, horizontal_flip=True introduces horizontal mirroring, and fill_mode='nearest' ensures that newly created pixels resulting from the transformations are populated using the nearest available pixel value. This comprehensive set of geometric transformations significantly expands the effective training data manifold, encouraging the model to learn features invariant to minor changes in image pose and scale and to expose the CNN to a wider variety of images than it has seen before, teaching it to recognize the core objects and patterns regardless of their position, orientation, scale, or lighting conditions .
- **Convolutional and Pooling**: The feature extraction component is constructed as a deep, three-block hierarchical structure designed to capture features from edge details to complex patterns.The layer takes an input image (or volume of feature maps) and slides a small, weighted matrix—the kernel (or filter)—across its spatial dimensions. At every position, it performs a matrix multiplication between the filter's weights and the corresponding input pixels, producing a single output value.
  Each of the three blocks strictly follows the sequence: Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPooling2D $\rightarrow$ Dropout.This repeating pattern is a fundamental design choice in CNNs to iteratively extract features.Pooling is applied after each convolutional block to downsample the feature maps. The $2 \times 2$ pool size, combined with a stride of 2 (the default for MaxPooling2D), reduces both the height and width of the feature map by half, resulting in a $75\%$ reduction in spatial dimensionality. The Max Pooling function operates by selecting only the maximum activation within the $2 \times 2$ local window meaning it reports the maximum output within a rectangular neighborhood (summarizes the responses) and helps to make the representation approximately invariant to small translations of the input.
- **Kernel and Padding**: All layers use a $3 \times 3$ kernel size, chosen for its optimal balance between computational efficiency and receptive field size, efficiently capturing local pixel relationships. The kernel convolves across the width and height of the input image in the first layer and performs element-wise multiplication with the corresponding pixels of its size. Then sums the results and produces a **single pixel** in the output. Each kernel learns to detect a specific, low-level feature in the image. The use of 'same' padding is crucial: it ensures that the spatial output dimension (height $\times$ width) of the convolution layer matches its input dimension, thereby preserving information density before downsampling by the pooling layers. Without padding the spatial dimensions (height and width) of the output feature map would **shrink**.
- **Filter Scaling**:The filter count is aggressively scaled across the depth of the network: 65, 130, and 258 filters. This exponential increase allows the network to dedicate a greater capacity (more filters) to capturing the highly abstract and synthesized features in the deeper layers.The shallow layer with 65 filters primarily learn basic, low-level features while the Deep Layers with 130 and 258 filters integrate and synthesize high-level features and complex patterns.
- **Activation and Initialization**: Rectified Linear Unit

(ReLU) is used for its effectiveness in introducing non-linearity without suffering from the vanishing gradient problem, which is common in older activations. The use of 'he_normal' kernel initialization is perfectly paired with ReLU, as it initializes weights to maintain activation variance through the layers.Otherwise, if the initial weights of the kernels are too large or too small, either activations will grow rapidly layer after layer, leading to exploding gradients during backpropagation or shrink toward zero, causing the $\max(0, x)$ part of ReLU to output zero for most inputs. This results in dead neurons and vanishing gradients, stalling the learning process.

- **Dropout**: This layer after each pooling step performs local regularization, temporarily nullifying $20\%$ of the feature map activations. This ensures that the subsequent convolutional layer is forced to learn robust feature combinations that do not rely on a few specific preceding neurons.
- **Global Average Pooling**: This layer serves as the crucial link between the convolutional base and the dense classification layers. It takes the output of the last convolutional layer (which has a shape of $W \times H \times D$, where $W$ is width, $H$ is height, and $D$ is the depth/number of feature maps) and calculates the average value for each $W \times H$ feature map. The output is a $1D$ vector, meaning it averages each 2D feature map into a single scalar Deep Feature Vector ($258 \times 1$), which is the number of feature maps. Significantly reduces the total number of parameters, making the model less prone to overfitting and often allowing for better localization of features learned by the CNN and acts as a superior regularization technique.It doesn't matter exactly where a feature is detected; only that it was detected.

By replacing the CNN's classification head (**Softmax**) with an SVM, we shift the objective from minimizing cross-entropy which is a probabilistic loss, to maximizing the separation margin, thus geometric loss. The Radial Basis Function $K(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2)$ maps the extracted features into a **Reproducing Kernel Hilbert Space** of infinite dimensions. In this space, complex, non-linear relationships in the original feature set become linearly separable.

### *Training Analysis and Evaluation*

The classification results demonstrate a performance plateau at 82.54% accuracy, marking a significant elevation in the model's discriminative capacity compared to traditional non-convolutional approaches. By replacing the standard Softmax output with a Support Vector Machine utilizing a Radial Basis Function kernel, the architecture transitions from a probabilistic objective to one centered on structural risk minimization. The CNN backbone functions as a non-linear mapping function $\phi$, transforming raw $32 \times 32 \times 3$ pixel tensors into a high-dimensional manifold where semantic structures are better isolated. Then the Global Average Pooling layer serves as a bottleneck, collapsing spatial dimensions to a 258-element vector. This hybrid framework successfully leverages translation invariance from the convolutional layers and margin maximization from the SVM.

```
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
SVM Model Accuracy: 0.8254

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.86      0.84      1000
           1       0.92      0.91      0.92      1000
           2       0.76      0.74      0.75      1000
           3       0.66      0.67      0.66      1000
           4       0.81      0.82      0.82      1000
           5       0.75      0.75      0.75      1000
           6       0.86      0.86      0.86      1000
           7       0.86      0.85      0.86      1000
           8       0.91      0.91      0.91      1000
           9       0.89      0.89      0.89      1000

    accuracy                           0.83     10000
   macro avg       0.83      0.83      0.83     10000
weighted avg       0.83      0.83      0.83     10000
```

Fig. 8. SVM Accuracy and Classification Report

The classification report reveals a Macro Average of 0.83, indicating that the hybrid CNN-SVM framework has achieved a high degree of class-wise discriminability by mapping images into a well-separated feature space. The high F1-scores for Class 1 (0.92) and Class 8 (0.91) suggest that these categories possess distinct morphological features that the CNN extracts with high fidelity, allowing the RBF kernel to establish clear, wide-margin boundaries in the Reproducing Kernel Hilbert Space. Conversely, the performance trough observed in Class 3 (0.66) highlights a significant challenge in intra-class variance; here, the precision-recall balance suggests that the extracted feature vectors for this category likely overlap with other classes in the manifold, creating a region of topological ambiguity that even the local proximity mapping of the RBF kernel struggles to resolve. Overall, the consistent support of 1000 samples per class confirms that the model's 82.54% accuracy is a statistically robust measure of its ability to generalize across diverse visual distributions while remaining resilient to the noise typically inherent in raw pixel-level data.

### *Confusion Matrix Analysis*

The diagonal dominance of the matrix confirms a high degree of class-wise discriminability for the majority of the CIFAR-10 categories.

Optimal Clusters achieved in Classes 1 and 8, with F1-scores of 0.92 and 0.91 respectively. These classes represent the "upper bound" of the model's performance. The confusion matrix reveals that Class 1 Automobile and Class 8 Ship are rarely misclassified, suggesting the CNN extracts highly salient geometric features that the RBF kernel can easily isolate within the Reproducing Kernel Hilbert Space.

Semantic Confusion between Classes 3 and 5 is observed. The most significant performance degradation occurs in Class 3 Cat with an F1-score of 0.66. The matrix reveals a high degree of inter-class leakage between Class 3 and Class 5 Dog, with 125 instances of Class 3 being misclassified as Class 5, and 133 instances of Class 5 being misclassified as Class 3. This indicates a high topological proximity in the feature manifold, where the shapes of these two animal categories are semantically ambiguous.

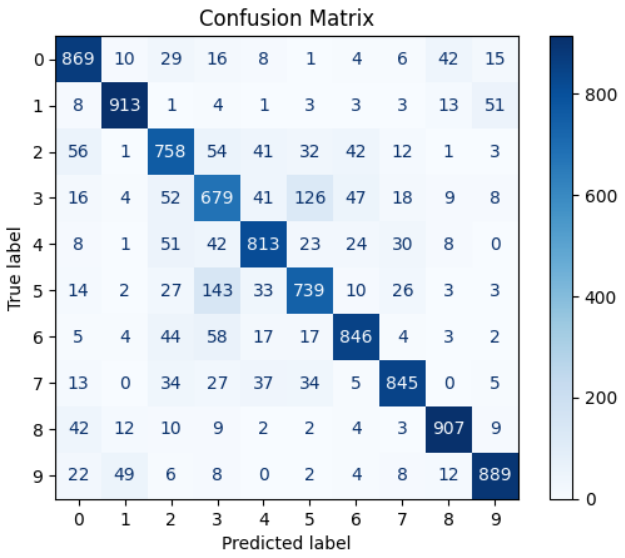



Fig. 9. Confusion Matrix for CNN-SVM.

## VIII. Comparison of Performance for Support Vector Machines and Machine Learning Algorithms

In the previous assignment we implemented two specific non-parametric, distance-based classifiers: the k-Nearest Neighbors (kNN) algorithm and the Nearest Centroid Classifier (NCC).The K-Nearest Neighbors algorithm operates on the principle of instance-based learning, classifying a new data point based on the majority class of its k closest neighbors in the training set and NCC calculates the mean image for every class (e.g., the average pixel value for all cat images) so in order to classify a new image, it calculates the distance to each class mean and chooses the closest one.The k-NN algorithm achieved 30% accuracy in the CIFAR-10 dataset,thus k-NN is only picking up on very coarse similarities in color and lighting, but it completely fails to understand concepts like edges, corners, or object structures.On the other hand, NCC's 27% accuracy reflects the fact that a mean image is a horrible representation of a complex class. The average "cat" image is blurry and contains none of the specific features needed to distinguish it from an average "dog" or "deer" image.It throws away all the meaningful variance required for discrimination. The mean vector destroys all variance and crucial distinguishing details, making it almost useless for classification.Even though k-NN is slightly better because it uses individual instances, not the average. A CIFAR-10 image is $32 \times 32 \times 3$ (RGB), resulting in a vector of 3,072 dimensions. In this high-dimensional space, the distance between data points becomes unreliable (known as the "curse of dimensionality").

### *Transition to SVM*

The transition from distance-based methods like kNN to Support Vector Machines (SVM) represents a shift from simply finding neighbors to finding optimal decision boundaries. While kNN and NCC struggle with the "curse of dimensionality" in the 3,072-dimensional space of CIFAR-10, SVMs utilize the One-vs-Rest approach to efficiently manage the 10-class problem by training only 10 models. The choice of kernel significantly impacts how these boundaries are drawn in high-dimensional space.For the **Linear Kernel** that achieved 40.65% accuracy, while an improvement over the 27-30% is seen in NCC and kNN, the linear kernel is still restricted to drawing straight decision boundaries. This performance level suggests that while some classes are linearly separable based on global color or contrast, the majority of the CIFAR-10 categories require a more flexible approach to define their boundaries. With **Polynomial Kernel** there is a significant increase to 54% that reflects the kernel's ability to model non-linear interactions between pixels. By projecting the 3,072 dimensions into a higher-dimensional space, the polynomial kernel can identify shapes and patterns that a linear line would miss, effectively overcoming some of the distance unreliability. . Moving from a Polynomial kernel to an RBF kernel with 57% accuracy provides a performance boost by allowing the model to create localized, non-linear decision boundaries. The most significant leap in accuracy, however, comes from using the Histogram of Oriented Gradients (HOG) feature descriptor, which brings the model to 64%. This jump occurs because HOG transforms the raw, 3,072-dimensional pixel vector—which previously caused kNN and NCC to fail due to their sensitivity to lighting and color—into a representation focused on local shapes and edge orientations. By calculating the distribution of gradient directions in small cells of the image, HOG captures the structural essence of objects, such as the wheels of a car or the wings of a bird. When these structurally-rich features are fed into the SVM with RBF Kernel, the model no longer struggles with the blurry representations of NCC or the coarse color-matching of kNN, but instead learns to distinguish classes based on the actual geometric properties of the objects. Lastly, the combination of Convolutional Neural Networks and SVM represents the pinnacle of this classification progression, achieving a significant accuracy of 82.54%. In this framework, the convolutional layers act as a high-powered feature extractor that replaces manual methods like HOG, automatically learning to identify complex geometric structures and spatial hierarchies within the 3,072-dimensional image space. By integrating an SVM as the final classifier, the model moves away from probabilistic predictions to focus on maximizing the classification margin, which provides more robust decision boundaries in high-dimensional space.

### *Conclusion*

The progression of CIFAR-10 classification models illustrates a clear shift from simple, distance-based metrics to advanced, feature-learned architectures that overcome the inherent limitations of high-dimensional data. Initial non-parametric models like kNN (30%) and NCC (27%) struggled significantly because raw pixel distances in a 3,072-dimensional space are mathematically unreliable—a phenomenon known as the curse of dimensionality—and mean vectors effectively destroy the variance and structural details needed to distinguish complex classes. Performance improved as Support Vector Machines (SVM) were introduced, utilizing Linear (40%),

Polynomial (54%), and RBF (57%) kernels to bend decision boundaries and capture non-linear relationships that simple proximity metrics could not. This efficiency was maintained through the One-vs-Rest approach, which was specifically selected over the One-vs-One strategy for CIFAR-10 because it optimizes computational resources by training only 10 models. While manual feature extraction like HOG (64%) improved results by focusing on edge orientations, the peak accuracy of 82.54% achieved by the CNN-SVM hybrid demonstrates that automated hierarchical feature learning is the most effective solution for capturing semantic object structures.